Artificial Intelligence

Chapter 5: Neural Networks and Deep Learning Algorithms

Lecture by
Kiran Bagale

IoE, Thapathali Campus

Feb, 2026

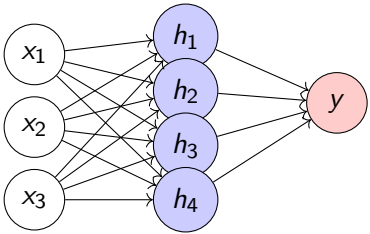
Outline

Biological Inspiration:

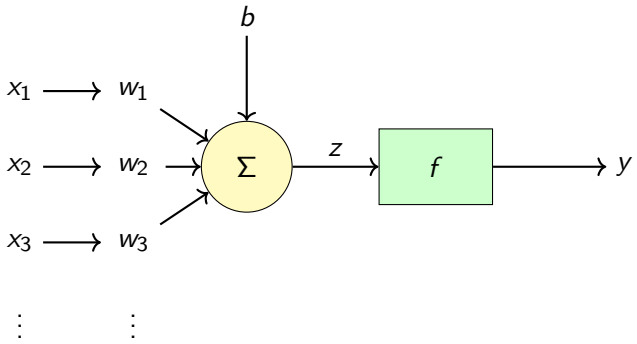
- Inspired by neurons in the brain
- Networks of interconnected processing units
- Learn from experience through training

Mathematical Model:

- Nodes (neurons) connected by weighted edges
- Process inputs through activation functions
- Produce outputs for decision making



Structure of a Single Neuron



Mathematical Formulation:

$$z = \sum_{i=1}^{n} w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

Common Activation Functions

1. Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Range: $(0, 1)$
- Smooth gradient
- Output interpretable as probability
- Issue: Vanishing gradients

2. Hyperbolic Tangent (tanh)

$$\tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

3. ReLU (Rectified Linear Unit)

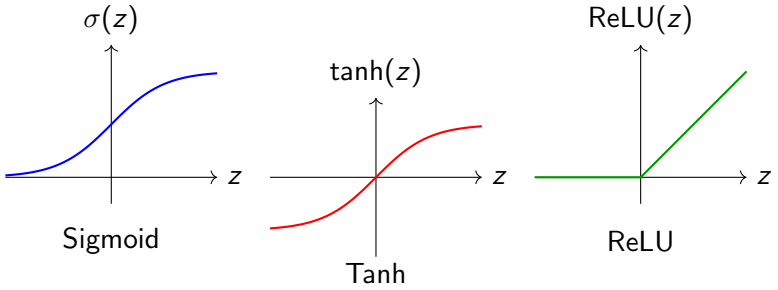
$$\text{ReLU}(z) = \max(0, z)$$

- Most popular in deep learning
- Computationally efficient
- Mitigates vanishing gradient
- Issue: Dead neurons

4. Leaky ReLU

$$\text{LReLU}(z) = \max(\alpha z, z), \quad \alpha \approx 0.01$$

Activation Functions Visualization



Network Architectures

Feedforward Neural Network:

- Information flows in one direction: input $\rightarrow$ hidden $\rightarrow$ output
- No cycles or loops
- Most common architecture

Key Components:

- **Input Layer**: Receives input features
- **Hidden Layers**: Perform computations and feature extraction
- **Output Layer**: Produces final predictions
- **Weights and Biases**: Learnable parameters

Notation:

- L: Number of layers
- $n^{[l]}$: Number of units in layer l
- $\mathbf{W}^{[l]}$: Weight matrix for layer l
- $\mathbf{b}^{[l]}$: Bias vector for layer l

The Perceptron

Simplest Neural Network Model (Rosenblatt, 1958)

Binary Classification:

$$y = \begin{cases} 1 & \text{if } \mathbf{w}^T\mathbf{x} + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

Learning Algorithm:

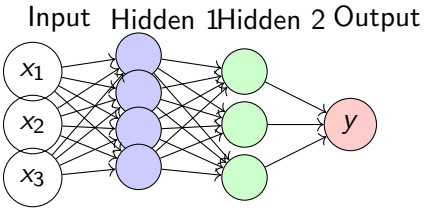
1. Initialize weights randomly
2. For each training example $(\mathbf{x}^{(i)}, y^{(i)})$:
   - Compute prediction: $\hat{y}^{(i)}$
   - Update weights: $\mathbf{w} := \mathbf{w} + \eta(y^{(i)} - \hat{y}^{(i)})\mathbf{x}^{(i)}$
   - Update bias: $b := b + \eta(y^{(i)} - \hat{y}^{(i)})$
3. Repeat until convergence

Limitation: Can only learn linearly separable functions (e.g., cannot learn XOR)

Multilayer Perceptron (MLP)

Overcomes perceptron limitations with hidden layers
Architecture:

- Multiple layers of neurons
- Each layer fully connected to next
- Non-linear activation functions
- Universal function approximator



Forward Propagation:

$$\mathbf{z}^{[l]} = \mathbf{W}^{[l]}\mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}$$
$$\mathbf{a}^{[l]} = f^{[l]}(\mathbf{z}^{[l]})$$

where $\mathbf{a}^{[0]} = \mathbf{x}$ (input)

Backpropagation Algorithm

Efficient method for computing gradients (Rumelhart et al., 1986)

Key Idea: Use chain rule to compute gradients layer by layer, from output to input

Loss Function: For regression or classification

$$\mathcal{L}(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2 \quad \text{or} \quad \mathcal{L}(\hat{y}, y) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

Gradient Computation:

1. **Output layer:** $\delta^{[L]} = \nabla_{\mathbf{a}^{[L]}} \mathcal{L} \odot f'^{[L]}(\mathbf{z}^{[L]})$
2. **Hidden layers:** $\delta^{[l]} = (\mathbf{W}^{[l+1]T} \delta^{[l+1]}) \odot f'^{[l]}(\mathbf{z}^{[l]})$
3. **Parameter gradients:**

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}} = \delta^{[l]} \mathbf{a}^{[l-1]T}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[l]}} = \delta^{[l]}$$

Backpropagation: Step by Step

Input: Training example $(\mathbf{x}, y)$, network parameters

Forward Pass:

for $l = 1$ to L **do**

$\mathbf{z}^{[l]} = \mathbf{W}^{[l]}\mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}$

$\mathbf{a}^{[l]} = f^{[l]}(\mathbf{z}^{[l]})$

end for

Compute loss: $\mathcal{L} = \mathcal{L}(\mathbf{a}^{[L]}, y)$

Backward Pass:

Compute output gradient:
$\delta^{[L]} = \nabla_{\mathbf{a}^{[L]}}\mathcal{L} \odot f'^{[L]}(\mathbf{z}^{[L]})$

for $l = L - 1$ down to 1 **do**

$\delta^{[l]} = (\mathbf{W}^{[l+1]T}\delta^{[l+1]}) \odot f'^{[l]}(\mathbf{z}^{[l]})$

$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}} = \delta^{[l]}\mathbf{a}^{[l-1]T}$

$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[l]}} = \delta^{[l]}$

end for

Update parameters:

$\mathbf{W}^{[l]} := \mathbf{W}^{[l]} - \eta\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}}$, $\mathbf{b}^{[l]} := \mathbf{b}^{[l]} - \eta\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[l]}}$

Training Neural Networks

Gradient Descent Variants:

- **Batch Gradient Descent:** Use entire dataset for each update

$$\mathbf{W} := \mathbf{W} - \eta \frac{1}{m} \sum_{i=1}^{m} \nabla_{\mathbf{W}} \mathcal{L}^{(i)}$$

- **Stochastic Gradient Descent (SGD):** Use one example at a time

$$\mathbf{W} := \mathbf{W} - \eta \nabla_{\mathbf{W}} \mathcal{L}^{(i)}$$

- **Mini-batch Gradient Descent:** Use small batches (e.g., 32, 64, 128)

$$\mathbf{W} := \mathbf{W} - \eta \frac{1}{b} \sum_{i \in \text{batch}} \nabla_{\mathbf{W}} \mathcal{L}^{(i)}$$

Challenges:

- Choosing learning rate η
- Avoiding local minima and saddle points
- Vanishing/exploding gradients
- Overfitting

What is Deep Learning?

Deep Learning = Neural Networks with Many Layers

Characteristics:

- Multiple hidden layers (deep architectures)
- Learns hierarchical representations
- Automatic feature extraction
- End-to-end learning
- Requires large datasets
- Computationally intensive

Why Deep?

- Each layer learns increasingly abstract features
- Layer 1: Edges, corners
- Layer 2: Textures, patterns
- Layer 3: Object parts
- Layer 4: Objects

Breakthrough Factors:

- Big data availability
- GPU computing power
- Better algorithms

Deep Learning vs Traditional ML

Aspect	**Traditional ML**	**Deep Learning**
Feature Engineering	Manual, domain expertise	Automatic, learned
Data Requirements	Works with small datasets	Requires large datasets
Architecture	Shallow models	Deep architectures
Training Time	Faster	Slower, GPU-intensive
Interpretability	More interpretable	Black box
Performance	Good for structured data	Excellent for unstructured data
Examples	SVM, Decision Trees	CNN, RNN, Transformers

Popular Deep Learning Architectures

1. Convolutional Neural Networks (CNNs)

- Specialized for image processing
- Uses convolutional layers, pooling layers
- Applications: Image classification, object detection, segmentation

2. Recurrent Neural Networks (RNNs)

- Designed for sequential data
- Maintains hidden state across time steps
- Applications: Language modeling, machine translation, speech recognition

3. Transformers

- Attention-based architecture
- Parallel processing of sequences
- Applications: NLP, GPT models, BERT

Regularization Techniques

Preventing Overfitting in Deep Networks

1. Dropout

- Randomly deactivate neurons during training
- Prevents co-adaptation of features
- Typical rate: 0.5 for hidden layers

2. L1/L2 Regularization

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda \sum_{l} ||\mathbf{W}^{[l]}||^2$$

3. Batch Normalization

- Normalize layer inputs
- Accelerates training
- Reduces internal covariate shift

Optimization Algorithms

Beyond Standard SGD:

1. Momentum

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + (1-\beta)\nabla_{\mathbf{W}}\mathcal{L}$$
$$\mathbf{W} := \mathbf{W} - \eta \mathbf{v}_t$$

2. RMSprop

$$\mathbf{s}_t = \beta \mathbf{s}_{t-1} + (1-\beta)(\nabla_{\mathbf{W}}\mathcal{L})^2$$
$$\mathbf{W} := \mathbf{W} - \eta \frac{\nabla_{\mathbf{W}}\mathcal{L}}{\sqrt{\mathbf{s}_t + \epsilon}}$$

3. Adam (Adaptive Moment Estimation)

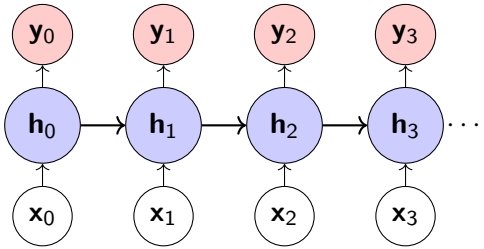
- Combines momentum and RMSprop
- Most popular optimizer for deep learning
- Adapts learning rate for each parameter

Recurrent Neural Networks (RNNs)

Networks with Memory for Sequential Data

Key Features:

- Process sequences of variable length
- Maintain hidden state
- Share parameters across time steps
- Capture temporal dependencies



RNN Equations:

$$\mathbf{h}_t = f(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$$

$$\mathbf{y}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y$$

Applications:

- Language modeling, machine translation
- Speech recognition, video analysis, time series prediction

Challenges with Standard RNNs

1. Vanishing Gradient Problem

- Gradients become very small during backpropagation through time
- Network cannot learn long-term dependencies
- Affects layers far from output

2. Exploding Gradient Problem

- Gradients become very large
- Causes numerical instability
- Solution: Gradient clipping

Solutions:

- **LSTM (Long Short-Term Memory)**: Special architecture with gates
- **GRU (Gated Recurrent Unit)**: Simplified version of LSTM
- Both can capture long-range dependencies

Long Short-Term Memory (LSTM)

Sophisticated RNN Architecture with Memory Cells
Key Components:

- **Cell state $\mathbf{c}_t$**: Long-term memory
- **Hidden state $\mathbf{h}_t$**: Short-term memory
- **Forget gate**: What to remove
- **Input gate**: What to add
- **Output gate**: What to output

Advantages:

- Learns long-term dependencies
- Mitigates vanishing gradients
- Selective memory retention

LSTM Equations:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$
$$\mathbf{i}_t = \sigma(\mathbf{W}_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$
$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$$
$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$
$$\mathbf{o}_t = \sigma(\mathbf{W}_o \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$
$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

Generative Neural Networks

Networks that Generate New Data 1. Autoencoders

- Encoder: Compress input to latent representation
- Decoder: Reconstruct input from latent code
- Applications: Dimensionality reduction, denoising, anomaly detection

2. Variational Autoencoders (VAEs)

- Probabilistic version of autoencoders
- Learn distribution over latent space
- Generate new samples by sampling from latent distribution

3. Generative Adversarial Networks (GANs)

- Two networks: Generator and Discriminator
- Adversarial training process
- State-of-the-art image generation

Generative Adversarial Networks (GANs)

Architecture:

- **Generator** G: Creates fake data from noise

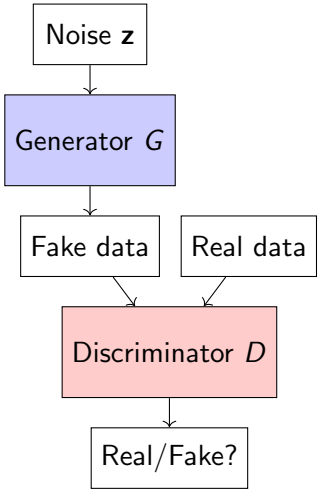
$$\mathbf{x}_{\text{fake}} = G(\mathbf{z}), \quad \mathbf{z} \sim p_z(\mathbf{z})$$

- **Discriminator** D: Distinguishes real from fake

$$D(\mathbf{x}) \in [0, 1]$$

Training Objective:

$$\min_G \max_D \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z}[\log(1 - D(G(\mathbf{z})))]$$



Applications: Image generation, style transfer, data augmentation, super-resolution

Applications of Deep Learning

Computer Vision:

- Image classification
- Object detection
- Semantic segmentation
- Face recognition
- Medical image analysis

Natural Language Processing:

- Machine translation
- Sentiment analysis
- Text generation
- Question answering
- Chatbots

Speech and Audio:

- Speech recognition
- Text-to-speech
- Music generation
- Audio classification

Other Domains:

- Game playing (AlphaGo)
- Autonomous driving
- Recommendation systems
- Drug discovery
- Financial forecasting

Key Takeaways

1. **Neural networks** are inspired by biological neurons and consist of interconnected layers with activation functions
2. **Multilayer perceptrons** overcome single perceptron limitations and learn complex non-linear functions through backpropagation
3. **Deep learning** uses many layers to learn hierarchical representations automatically from data
4. **Recurrent networks** process sequential data and maintain memory through hidden states
5. **Generative models** like GANs and VAEs can create new data samples by learning data distributions
6. Success requires: large datasets, computational resources, proper regularization, and good optimization techniques

References

- Russell, S., Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*. Pearson.
- Rich, E., Knight, K., Nair, S. B. (2009). *Artificial Intelligence*. McGraw-Hill.
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press.

Thank You!

Questions?